QuillAudits

AUDIT REPORT

March 2026

For

kyrrex

Table of Content

Executive Summary

Project Name	Kyrrex V2
Protocol Type	EIP - 7702
Project URL	https://kyrrex.com/global
Overview	This contract implements an EIP-7702 based smart account execution module that enables authorized ETH and ERC20 transfers via off-chain signatures (EIP-712). It verifies signed transfer instructions, enforces nonce and deadline checks to prevent replay attacks, and executes transfers on behalf of the signer. The contract also supports batched execution of multiple transfers through a relayer or external caller.
Audit Scope	The scope of this Audit was to analyze the Kyrrex V2 Smart Contracts for quality, security, and correctness.
Source Code link	Zip file was provided
Contracts in Scope	eip7702.sol
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	2nd March - 4th March
Updated Code Received	19th March 2026
Review 2	26th March 2026
Fixed In	https://drive.google.com/drive/folders/11rQYaV_dA__kwELIjMS_7YdN--h6c_0E?usp=drive_link

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity

Critical	0(0.0%)
High	0(0.0%)
Medium	0(0.0%)
Low	0(0.0%)
Informational	3 (100.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	0	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Broken Nonce Validation Allows Unlimited Signature Replay	Informational	Resolved
2	Floating Pragma	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Informational Issues

Broken Nonce Validation Allows Unlimited Signature Replay

Resolved

Path

SafeEIP7702Impl.sol

Function Name

`sendETH(), sweepERC20()`

Description

Both sendETH and sweepERC20 implement a nonce check intended to prevent replay attacks, however the current logic does not enforce this correctly and may behave unexpectedly under certain conditions:

```
if (noncesAddresses[address(this)] == opNonce + 1) {
    revert Unauthorized();
}
```

This check only reverts when the stored nonce equals exactly opNonce + 1, rather than enforcing that the supplied opNonce matches the expected stored value. Any opNonce that does not satisfy this narrow equality will pass the check silently. Additionally, since the contract writes opNonce + 1 back to storage after execution, a caller supplying a lower-than-expected opNonce could regress the stored nonce to a smaller value.

Impact

Low

Likelihood

Low

Recommendation

Replace the current nonce check with either a simple auto-increment by removing the opNonce parameter entirely and calling noncesAddresses[address(this)]++ on each execution, or introduce a mapping(address => mapping(uint256 => bool)) public usedNonces to mark each nonce as consumed after use. The latter is preferred for batch scenarios where strict ordering is not required.

Floating Pragma

Resolved

Path

SafeEIP7702Impl.sol

Description

The contract uses a floating pragma declaration:

```
pragma solidity ^0.8.24;
```

The ^ symbol allows the contract to be compiled with any compiler version from 0.8.24 up to, but not including, 0.9.0. Different compiler versions may introduce behavioral differences, optimizations, or bug fixes that could affect contract execution in unintended ways.

Impact

Low

Likelihood

Low

Recommendation

Lock the pragma to a specific compiler version

Functional Tests

Some of the tests performed are mentioned below:

- Verified that a correctly signed EIP-712 message successfully executes ETH transfer to the intended recipient.
- Confirmed that a valid signature allows ERC20 tokens to be transferred to the specified address.
- Verified that transactions with invalid or tampered signatures correctly revert with the BadSig error.
- Confirmed that transfers fail when the provided deadline has already passed and revert with Expired.
- Verified that nonce protection prevents replay attacks by rejecting reused nonces.
- Confirmed that multiple transfers can be executed successfully in a single batch transaction.
- Verified that if any transfer fails inside a batch execution the transaction reverts with StepFailed.
- Confirmed that ERC20 transfer failures correctly revert with the TransferFailed error.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 interface	Interface	Calls transfer() on arbitrary ERC20 tokens via sweepERC20()	Token correctly implements ERC20 standard	Non-standard tokens (e.g. fee-on-transfer, rebasing) can cause silent failures or drain issues
EIP-7702 Delegation	Protocol-level	EOA delegates execution context to this contract (address(this) == EOA)	EOA has correctly set its delegation to this implementation	Malicious or incorrect delegation points EOA funds at wrong implementation
Solidity Compiler ^0.8.24	Tool	Compilation	Compiler has no critical bugs at this version	Version pinned with ^, allowing minor upgrades that may introduce subtle changes

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
SafeEIP7702Impl.sol	sendETH(address to, uint256 amount, uint256 opNonce, uint256 deadline, bytes sig)	**What this function can do** Transfers ETH from the delegated EOA to any to address after verifying a valid EIP-712 signature and nonce **What this function cannot/should not do** Transfer ETH without a valid signature; replay a previously used nonce; execute after deadline; accept a signature from anyone other than address(this) **Main invariant(s)** Signature must be from address(this); opNonce must not be consumed; block.timestamp ⩽ deadline **Access level** public callable by anyone, protected by signature verification

Contract Name	Function	Category
SafeEIP7702Impl.sol	sweepERC20(address token, address to, uint256 amount, uint256 opNonce, uint256 deadline, bytes sig)	**What this function can do** Calls transfer() on any ERC20 token to move tokens from the delegated EOA to to, after verifying signature and nonce **What this function cannot/should not do** Transfer tokens without a valid signature; reuse a spent nonce; interact with a token after deadline; accept signatures from addresses other than address(this) **Main invariant(s)** Signature must be from address(this); nonce must be unused; token transfer() must succeed; block.timestamp ⩽ deadline **Access level** public callable by anyone, protected by signature verification
SafeEIP7702Impl.sol	makeTransferExtended(TransferInfoExtended[] transfers)	**What this function can do** Iterates over an arbitrary array of transfer structs and calls sendETH or sweepERC20 on each from address; reverts the entire batch on any single failure **What this function cannot/should not do** Should not accept unbounded arrays (gas DoS risk); should not continue silently on step failure; should not allow arbitrary contract calls via from field **Main invariant(s)** Every step must succeed or whole batch reverts via StepFailed; each transfer is still protected by its own signature check **Access level** external fully open, no caller restriction

Contract Name	Function	Category
SafeEIP7702Impl.sol	makeTransferExtendedNative(TransferInfoExtended transfer)	**What this function can do** Calls sendETH on the from address for a single native ETH transfer using the provided struct **What this function cannot/should not do** Should not be used as a proxy to call arbitrary logic on from; does not validate that transfer.token is address(0) before routing **Main invariant(s)** sendETH on from must succeed; reverts with StepFailed(0, reason) on failure **Access level** external fully open, no caller restriction
SafeEIP7702Impl.sol	makeTransferExtendedToken(TransferInfoExtended transfer)	**What this function can do** Calls sweepERC20 on the from address for a single ERC20 transfer using the provided struct **What this function cannot/should not do** Should not sweep tokens from addresses that did not sign; does not enforce transfer.token ≠ address(0) before routing **Main invariant(s)** sweepERC20 on from must succeed; reverts with StepFailed(0, reason) on failure **Access level** external fully open, no caller restriction

Contract Name	Function	Category
SafeEIP7702Impl.sol	receive()	**What this function can do** Accepts plain ETH transfers sent directly to the contract; allows the delegated EOA to hold ETH that can be swept via sendETH **What this function cannot/should not do** Should not execute any logic beyond receiving ETH; cannot restrict who sends ETH to the address **Main invariant(s)** Contract balance increases by msg.value; no state is modified beyond the implicit balance update **Access level** external payable open to all senders

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
SafeEIP7702Impl.sol (as EIP-7702 delegated EOA)	Native ETH	ETH held directly in the delegated EOA's balance, spendable via sendETH with a valid signature
SafeEIP7702Impl.sol (as EIP-7702 delegated EOA)	ERC20 Tokens	Any ERC20 token balance held by the delegated EOA address, sweepable via sweepERC20 with a valid signature
Arbitrary ERC20 Token Contract (token param)	ERC20	Token balances are custodied inside the token contract itself; this contract only calls transfer() on it — it does not hold the tokens directly

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SafeEIP7702 Impl.sol	receive()	Native ETH	—	Anyone	Any address can deposit ETH into the EOA with no restriction or logging; no access control on who funds the wallet
SafeEIP7702 Impl.sol	sendETH()	—	Native ETH	Anyone (sig-gated)	Entire ETH balance can be drained in one call if a valid sig is produced; nonce check has a bug — uses == opNonce + 1 instead of >= opNonce + 1, allowing nonce reuse in edge cases
SafeEIP7702 Impl.sol	sweepERC20()	—	ERC20 Token	Anyone (sig-gated)	Same flawed nonce check as sendETH; non-standard tokens (fee-on-transfer, rebasing) may cause silent partial transfers or revert; ercrecover returning address(0) not explicitly rejected

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SafeEIP7702 Impl.sol	makeTransferExtended()	—	ETH or ERC20	Anyone	Unbounded array input enables gas DoS; arbitrary from addresses called with no allowlist; a single malicious from entry can cause full batch revert, blocking legitimate transfers
SafeEIP7702 Impl.sol	makeTransferExtendedNative()	—	Native ETH	Anyone	Delegates trust entirely to from address; no check that transfer.token == address(0) before routing to sendETH; misrouted calls may silently pass wrong parameters
SafeEIP7702 Impl.sol	makeTransferExtendedToken()	—	ERC20 Token	Anyone (sig-gated)	No check that transfer.token != address(0) before routing to sweepERC20; from address receives an arbitrary external call — reentrancy risk if from is a contract rather than a pure EIP-7702 EOA

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
External ERC20 Contract	transfer() (called internally)	—	ERC20 Token	SafeEIP7702Impl	Return value handling relies on both ok and decoded bool — non-compliant tokens that return nothing or revert unexpectedly can cause TransferFailed

Closing Summary

In this report, we have considered the security of Kyrrex V2. We performed our audit according to the procedure described above.

No critical issues in Kyrrex V2, just 2 issues of Informational severity were found, while both issues have been marked as resolved by the Kyrrex V2 team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

March 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com